

Chapter 17

Agentic Memory Systems

17.1 Motivation: Why Agents Need Memory

Large language models are, at their core, stateless function approximators: given a prompt x , they produce a distribution over continuations $p_\theta(y \mid x)$. Every inference call begins from scratch. The *context window*—the finite sequence of tokens the model can attend to—is the only information available at generation time. For short, self-contained tasks this is sufficient. For long-horizon agentic tasks it is a fundamental bottleneck.

The Context-Window Bottleneck

Let L denote the maximum context length (e.g. $L = 128,000$ tokens for GPT-4o). A single token encodes roughly 4 characters; a typical book contains $\sim 500,000$ words $\approx 670,000$ tokens. Even ignoring cost, a multi-day autonomous agent accumulates observations, tool outputs, and reasoning traces that *cannot* fit in any fixed window. Memory systems are the engineering response to this physical constraint.

Three distinct failure modes arise when agents lack persistent memory:

1. **Catastrophic forgetting of context.** Once an event scrolls out of the context window it is irrecoverably lost. The agent cannot refer back to a decision made 10,000 tokens ago.
2. **Inability to learn from experience.** Without episodic storage, every episode is the agent's first. Successful strategies cannot be reused; mistakes are repeated.
3. **Lack of personalization.** User preferences, domain facts, and relationship history must be re-established in every session, degrading user experience and efficiency.

Memory as Cognitive Architecture

Cognitive science distinguishes several memory systems in biological agents [337, 356]: *working memory* (active manipulation of information), *episodic memory* (autobiographical events), *semantic memory* (world knowledge), and *procedural memory* (skills and habits). Effective agentic AI systems benefit from analogous distinctions—not because we are simulating neuroscience, but because these categories reflect genuinely different *access patterns*, *update frequencies*, and *retrieval mechanisms*.

Formally, we model an agent as a tuple $\mathcal{A} = (\pi_\theta, \mathcal{M}, \mathcal{R}, \mathcal{W})$ where π_θ is the policy (the LLM), $\mathcal{M}$ is the memory store, $\mathcal{R} : \mathcal{Q} \times \mathcal{M} \rightarrow \mathcal{D}$ is a retrieval function mapping queries to retrieved documents, and $\mathcal{W} : \mathcal{M} \times \mathcal{E} \rightarrow \mathcal{M}$ is a write function updating memory with new experiences $\mathcal{E}$. At each step t the agent observes o_t , retrieves relevant context $c_t = \mathcal{R}(o_t, \mathcal{M})$, and acts:

$$a_t \sim \pi_\theta(\cdot \mid [s_t; c_t; h_t]),$$

where s_t is the current system prompt, c_t is retrieved memory, and h_t is the recent in-context history. After acting, the agent may write new information: $\mathcal{M} \leftarrow \mathcal{W}(\mathcal{M}, (o_t, a_t, r_t))$.

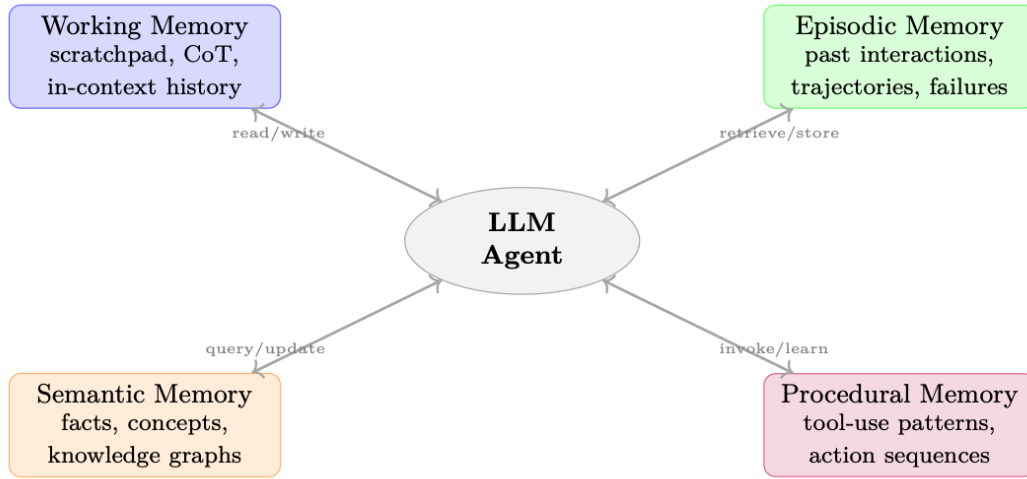


Figure 17.1: Four-way taxonomy of agentic memory systems, mirroring cognitive science distinctions. Each memory type has distinct access patterns, update frequencies, and retrieval mechanisms.

17.2 Taxonomy of Memory Types

17.2.1 Working Memory (Short-Term)

Working memory is the agent’s *active workspace*: the information currently being manipulated. In LLM agents it corresponds to:

- **Scratchpads.** Intermediate reasoning steps written to a dedicated buffer before producing a final answer (e.g. chain-of-thought [374], scratchpad [272]).
- **Chain-of-thought buffers.** The sequence of reasoning tokens $z_1, z_2, \dots, z_k$ generated before the answer token a , modeled as $p(a | x) = \sum_z p(a | x, z) p(z | x)$.
- **Conversation context.** The recent turn history $[(u_1, a_1), \dots, (u_t, a_t)]$ kept in the context window.

Working memory is *fast* (zero retrieval latency—it is already in context), *volatile* (lost when the context is cleared), and *capacity-limited* (bounded by L).

17.2.2 Episodic Memory (Experience-Based)

Episodic memory stores *specific past events* indexed by context and time. For agents:

- **Past interactions.** Full or summarized records of prior conversations, task attempts, and their outcomes.
- **Successful trajectories.** High-reward action sequences that can be retrieved as few-shot exemplars for similar future tasks.
- **Failure cases.** Documented mistakes with root-cause annotations, enabling the agent to avoid repeating errors.
- **Retrieval-augmented episodic recall.** Given a new task q , retrieve the k most similar past episodes $\{e_i\}_{i=1}^k$ and include them in context.

Episodic memory is typically implemented as a vector store (Section 17.3.1) with embeddings over episode summaries.

17.2.3 Semantic Memory (World Knowledge)

Semantic memory encodes *general facts and concepts* decoupled from specific episodes:

- **Factual knowledge.** Entities, attributes, and relationships (e.g. “Paris is the capital of France”).
- **Domain concepts.** Definitions, taxonomies, and ontologies relevant to the agent’s task domain.
- **Knowledge graphs.** Structured representations $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where nodes $v \in \mathcal{V}$ are entities and edges $e \in \mathcal{E}$ are typed relations.

Unlike episodic memory, semantic memory is *context-independent*: the fact that water boils at 100°C is true regardless of when or where it was learned.

17.2.4 Procedural Memory (Skills)

Procedural memory encodes *how to do things*—skills and action patterns that have been automatized:

- **Learned tool-use patterns.** Which API to call for which task, how to format inputs, how to handle errors.
- **Action sequences.** Multi-step procedures (e.g. “to deploy code: run tests → build image → push → update manifest”).
- **Policies as memory.** The model weights θ themselves encode procedural knowledge; fine-tuning on successful trajectories is a form of procedural memory consolidation.

Memory Type Classification

An agent helping with software development uses:

- **Working:** the current file being edited, the error message just received.
- **Episodic:** “Last week I fixed a similar `NullPointerException` in module X by adding a null check at line 42.”
- **Semantic:** “Python’s `asyncio.gather` runs coroutines concurrently; exceptions propagate unless `return_exceptions=True`.”
- **Procedural:** the standard debugging workflow: reproduce → isolate → hypothesize → test → fix.

17.3 Memory Architectures

17.3.1 RAG-Based Memory

Retrieval-Augmented Generation (RAG) [208] is the dominant paradigm for external memory in LLM agents. The memory store $\mathcal{M}$ is a collection of documents $\{d_i\}_{i=1}^N$; retrieval maps a query q to a ranked subset.

Embedding Stores and Vector Databases. Each document d_i is encoded by an embedding model ϕ : $\mathbf{v}_i = \phi(d_i) \in \mathbb{R}^D$. Queries are similarly encoded: $\mathbf{q} = \phi(q)$. Retrieval returns the top- k documents by similarity:

$$\text{Retrieve}(q, \mathcal{M}, k) = \arg \max_{S \subseteq [N], |S|=k} \sum_{i \in S} \text{sim}(\mathbf{q}, \mathbf{v}_i),$$

where $\text{sim}(\cdot, \cdot)$ is typically cosine similarity. Approximate nearest-neighbor (ANN) indices (FAISS [171], HNSW [248], ScaNN [126]) make this tractable for $N \sim 10^7$.

Retrieval Strategies.

- **Dense retrieval.** Both query and documents are encoded by neural encoders (e.g. DPR [180], `text-embedding-3-large`). Captures semantic similarity but requires GPU inference.
- **Sparse retrieval.** BM25 or TF-IDF over token overlap. Fast, interpretable, strong for exact keyword matches.
- **Hybrid retrieval.** Combine dense and sparse scores via reciprocal rank fusion (RRF):

$$\text{RRF}(d, k) = \sum_{r \in \text{rankers}} \frac{1}{k + \text{rank}_r(d)},$$

where $k = 60$ is a smoothing constant. Hybrid consistently outperforms either alone [44].

Re-ranking. A cross-encoder re-ranker $f_\psi(q, d) \in [0, 1]$ scores each retrieved document jointly with the query, providing higher accuracy at the cost of $O(k)$ forward passes. The pipeline is: retrieve $k' \gg k$ candidates with ANN, re-rank with cross-encoder, return top k .

Retrieval Hallucination Risk

RAG does not eliminate hallucination—it can *introduce* it. If the retrieved document is outdated, incorrect, or only superficially relevant, the model may confidently incorporate false information. Always include provenance metadata (source, timestamp, confidence) and consider faithfulness verification steps.

17.3.2 Summarization-Based Memory

When verbatim storage is too expensive or noisy, *summarization* compresses information before storage.

Progressive Summarization. At each step t , the agent maintains a running summary S_t . When new information e_t arrives:

$$S_{t+1} = \text{LLM}(\text{"Summarize: } [S_t] + [e_t]\text{"}).$$

This keeps memory size $O(1)$ but risks losing detail.

Hierarchical Compression. Organize memory in levels $L_0 \supset L_1 \supset \dots \supset L_K$ where L_0 is verbatim and each L_{i+1} is a summary of L_i . Retrieval first checks L_K (most compressed, fastest) and drills down as needed. This mirrors the *progressive summarization* technique of Forte [96].

When to Summarize vs. Store Verbatim.

- Store verbatim: precise facts, code snippets, numerical results, user quotes.
- Summarize: narrative context, reasoning chains, redundant observations.
- Discard: noise, failed tool calls with no informational content.

17.3.3 Graph-Based Memory

Knowledge Graphs. A knowledge graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{R})$ stores facts as triples (h, r, t) where $h, t \in \mathcal{V}$ are entities and $r \in \mathcal{R}$ is a relation. Agents can query via SPARQL [134], Cypher [97], or natural-language-to-graph translation.

Entity-Relation Extraction. New observations are parsed by an extraction model $\text{IE} : \text{text} \rightarrow \{(h_i, r_i, t_i)\}$ and merged into $\mathcal{G}$. Coreference resolution and entity linking ensure consistency.

GraphRAG. GraphRAG [82] augments RAG with graph traversal: given a query, retrieve seed entities, then expand via k -hop neighborhood traversal to surface related facts not directly matched by embedding similarity. This is particularly powerful for multi-hop reasoning:

$$\text{GraphRetrieve}(q, \mathcal{G}, k) = \bigcup_{v \in \text{seeds}(q)} \mathcal{N}_k(v, \mathcal{G}),$$

where $\mathcal{N}_k(v, \mathcal{G})$ is the k -hop neighborhood of v .

Temporal Knowledge Graphs. Facts have validity intervals: $(h, r, t, [t_{\text{start}}, t_{\text{end}}])$. Temporal KGs [197] enable queries like “Who was the CEO of OpenAI in 2023?” without conflating past and present states.

17.3.4 Key-Value Memory Networks

Differentiable memory networks [342, 377] represent memory as a set of key-value pairs $\{(\mathbf{k}_i, \mathbf{v}_i)\}_{i=1}^M$ with soft attention-based retrieval:

$$\alpha_i = \text{softmax}\left(\frac{\mathbf{q}^\top \mathbf{k}_i}{\sqrt{D}}\right), \quad \mathbf{c} = \sum_{i=1}^M \alpha_i \mathbf{v}_i.$$

The retrieved context $\mathbf{c}$ is a differentiable function of the query, enabling end-to-end training. Modern transformer attention is a special case of this mechanism. For agentic use, memory slots can be updated via gradient descent or via explicit write operations.

17.3.5 MemGPT and Virtual Context Management

MemGPT [281] introduces a *virtual context* abstraction analogous to virtual memory in operating systems. Memory is organized in tiers:

Page-In / Page-Out Strategies. The agent decides *which* memory to promote to hot context (page-in) and *which* to evict (page-out) based on:

- **Recency:** recently accessed items are more likely to be needed.
- **Relevance:** items with high similarity to the current query.
- **Importance:** items tagged as high-importance during write.

Self-Directed Memory Management. In MemGPT, the LLM itself issues memory management function calls (`memory_search`, `memory_insert`, `memory_delete`) as part of its action space. This makes memory management a *learned behavior* rather than a hard-coded policy—a natural target for RL training (Section 17.7).

17.4 Memory Operations

17.4.1 Write: Committing to Memory

Not every observation should be stored. The write decision is a filtering problem:

$$\text{Write}(e) = \mathbf{1}[\text{importance}(e) > \tau],$$

where τ is a threshold and $\text{importance}(e)$ can be:

- **Surprise:** $-\log p_\theta(e \mid \text{context})$ —unexpected events are more informative.
- **Reward signal:** events associated with high $|r_t|$ (positive or negative) are worth remembering.
- **LLM self-assessment:** prompt the model to rate importance on a 1–10 scale.

Contradiction Detection. Before writing a new fact f_{new} , check for conflicts with existing memory:

$$\text{Conflict}(f_{\text{new}}, \mathcal{M}) = \exists f \in \mathcal{M} : \text{Contradicts}(f_{\text{new}}, f).$$

Contradiction detection can be implemented via NLI models or by prompting the LLM. On conflict, the agent must decide: overwrite, keep both with timestamps, or flag for human review.

Memory Format and Granularity. Beyond *what* to store, the *how* matters greatly. Memory entries range from atomic facts to verbose transcripts, with distinct trade-offs:

Table 17.1: Memory granularity trade-offs.

Format	Pros	Cons
Atomic facts “User prefers Python.”	Precise retrieval; composable; easy deduplication and contradiction detection	Loses context; extraction errors; brittle for nuanced information
Structured notes (A-MEM [400])	Rich metadata (tags, links); supports graph traversal; balances precision and context	Higher write cost; schema design required
Summarized episodes (MemGPT [281])	Preserves narrative coherence; compact; good for multi-turn reasoning	Summarization lossy; hard to update partially
Verbatim transcripts	Lossless; no extraction errors; supports exact quotation	Large storage; noisy retrieval; expensive to scan

In practice, production systems often combine granularities [53]: extract atomic facts for precise recall, maintain summarized episodes for narrative context, and archive verbatim transcripts in cold storage for auditability. The Generative Agents architecture [285] stores observations as atomic “memory objects” with natural-language descriptions, importance scores, and timestamps—enabling both precise retrieval and temporal reasoning.

Design Guidelines.

- **Match granularity to query type.** If users ask factoid questions (“What’s my API key?”), atomic facts win. If they ask contextual questions (“Why did we decide to use Redis?”), episode summaries are needed.
- **Store at the finest grain you can afford**, then build coarser views on top. It is easy to summarize atomic facts; it is impossible to recover atoms from a lossy summary.
- **Include provenance.** Every memory entry should link back to its source (conversation turn, document, tool output) so the agent can verify and the user can audit.

17.4.2 Read / Retrieve

Query Formulation. The retrieval query q need not be the raw observation. Better strategies:

- **HyDE (Hypothetical Document Embeddings) [106]:** generate a hypothetical answer, embed it, and use that embedding as the query.
- **Query expansion:** generate multiple paraphrases of the query and take the union of retrieved results.
- **Step-back prompting:** abstract the specific query to a more general question before retrieval.

Temporal Decay and Recency Bias. Older memories may be less relevant. A time-weighted score:

$$\text{score}(d, q, t) = \lambda \cdot \text{sim}(\mathbf{q}, \mathbf{v}_d) + (1 - \lambda) \cdot \exp\left(-\frac{t - t_d}{\tau_{\text{decay}}}\right),$$

where t_d is the memory’s creation time and τ_{decay} controls the decay rate. The Generative Agents paper [285] uses a similar recency-weighted retrieval.

17.4.3 Update: Conflict Resolution and Consolidation

Memory consolidation merges related memories to reduce redundancy and surface higher-level patterns:

$$\mathcal{M}' = \text{Consolidate}(\mathcal{M}) = \text{Cluster}(\mathcal{M}) \cup \text{Summarize}(\text{Cluster}(\mathcal{M})).$$

Forgetting Mechanisms. Biological memory forgets; so should artificial memory. Strategies:

- **LRU eviction:** remove least-recently-used entries when capacity is exceeded.
- **Importance-weighted forgetting:** $p(\text{forget} \mid d) \propto \exp(-\text{importance}(d))$.
- **Spaced repetition:** memories accessed repeatedly are retained longer, following the exponential forgetting curve [81].

17.4.4 Reflect: Meta-Cognitive Operations

Reflection [285, 326] is a higher-order memory operation: the agent reads its own memory and generates *insights*:

$$\text{Reflect}(\mathcal{M}) \rightarrow \{i_1, i_2, \dots\} \subset \mathcal{M}_{\text{semantic}},$$

where each insight i_j is a higher-level abstraction derived from multiple episodic memories.

Reflection in Practice (Reflexion)

After three failed attempts to solve a coding problem, the agent reflects:

1. Retrieves the three failure episodes from episodic memory.
2. Generates an insight: “I keep forgetting to handle the edge case where the input list is empty.”
3. Stores this insight in semantic memory.
4. On the next attempt, retrieves the insight and explicitly checks for empty inputs.

This is the core mechanism of Reflexion [326]: verbal reinforcement learning via self-reflection.

Where Do Reflections Live? Reflection *reads* from episodic memory but *writes* to semantic memory. The resulting insights are context-independent generalizations (“always check for empty inputs”), not episode-specific records—hence they belong in semantic memory $\mathcal{M}_{\text{semantic}}$. However, during the reflection process itself, the intermediate reasoning (retrieved episodes + synthesis prompt + generated insight) occupies *working memory* (the context window). In short:

- **Input:** episodic memory (specific past events)
- **Computation:** working memory (active reasoning in context)
- **Output:** semantic memory (durable, generalized insight)

This mirrors biological memory consolidation, where episodic experiences are gradually transformed into semantic knowledge during sleep and reflection.

17.5 Memory for Multi-Turn Conversations

17.5.1 User Modeling and Preference Tracking

A persistent user model $\mathcal{U}$ stores:

- **Explicit preferences:** stated likes/dislikes, communication style preferences.
- **Implicit preferences:** inferred from behavior (e.g. user always asks for code in Python, prefers concise answers).
- **Expertise level:** domain knowledge inferred from vocabulary and question complexity.
- **Goals and context:** ongoing projects, current tasks, organizational role.

The user model is updated after each interaction:

$$\mathcal{U}_{t+1} = \text{Update}(\mathcal{U}_t, (u_t, a_t, \text{feedback}_t)).$$

17.5.2 Session Continuity

Without memory, each conversation starts cold. With session memory:

1. At session start, retrieve the user model $\mathcal{U}$ and recent session summaries.
2. Inject a personalized system prompt: “You are helping Alice, a senior ML engineer working on a distributed training project. Last session you helped debug a gradient synchronization issue.”
3. At session end, summarize the session and update $\mathcal{U}$.

17.5.3 Personalization Through Memory

Personalization improves both *efficiency* (fewer clarifying questions) and *quality* (responses calibrated to user expertise). Key techniques:

- **Adaptive verbosity:** adjust response length based on user’s historical engagement.
- **Domain priming:** prepend relevant domain context from semantic memory.
- **Proactive recall:** surface relevant past interactions without being asked (“You asked about this topic last month; here’s what we found then”).

Privacy and Memory

Persistent user memory raises significant privacy concerns. Agents must: (1) obtain explicit consent before storing personal information, (2) provide mechanisms to inspect and delete stored memories, (3) enforce access controls in multi-user deployments, and (4) comply with data retention regulations (GDPR, CCPA). Memory systems should be designed with privacy-by-default.

17.6 Memory for Multi-Agent Systems

When multiple agents collaborate on a shared task, memory becomes a *coordination mechanism*—not just a personal knowledge store. A planning agent that decomposes a task must communicate sub-goals to executor agents; a critic agent must access the same context as the agent it evaluates; a research team of agents must avoid duplicating work. Without shared memory, agents must communicate everything through direct messages, creating bandwidth bottlenecks and losing information when conversations scroll out of context. Shared memory solves this by providing a persistent, queryable substrate that all agents can read from and write to—turning implicit coordination (“I hope the other agent remembers”) into explicit state (“the answer is on the blackboard”).

17.6.1 Shared Memory Pools

In multi-agent systems, agents may share a common memory store $\mathcal{M}_{\text{shared}}$ alongside private stores $\mathcal{M}_i$:

$$\text{context}_i(t) = \mathcal{R}(\mathcal{M}_i, q_i) \cup \mathcal{R}(\mathcal{M}_{\text{shared}}, q_i).$$

Shared memory enables *implicit coordination*: agent A writes a finding; agent B retrieves it without explicit communication.

17.6.2 Blackboard Architecture

The *blackboard* pattern [135] is a classic multi-agent coordination mechanism:

Each agent reads from and writes to the blackboard. A *controller* monitors the blackboard and activates agents when their preconditions are met. This decouples agents: they communicate through shared state rather than direct messaging.

17.6.3 Consensus and Conflict in Shared Knowledge

When multiple agents write to shared memory, conflicts arise. Resolution strategies:

- **Last-write-wins**: simple but loses information.
- **Versioned memory**: maintain a history of all writes; agents can query any version.
- **Voting / consensus**: require k -of- n agents to agree before a fact is committed.
- **Confidence-weighted merging**: $f_{\text{merged}} = \sum_i w_i f_i$ where w_i is agent i ’s confidence.
- **Designated authority**: assign ownership of memory regions to specific agents.

Open Problem: Distributed Memory Consistency

How should a multi-agent system maintain memory consistency under concurrent writes, network partitions, and adversarial agents? Classical distributed systems solutions (Paxos, Raft) apply but are expensive. Approximate consistency with bounded staleness may be sufficient for many agentic tasks—but the right trade-off is an open research question.

17.7 Training Memory Systems with Reinforcement Learning

17.7.1 Reward Signals for Memory Operations

Memory operations (read, write, update, reflect) can be treated as actions in the RL framework. The challenge is designing reward signals that incentivize *useful* memory behavior:

- **Task reward propagation**. If a memory retrieval leads to a correct answer, credit the retrieval action. Sparse but unambiguous.
- **Retrieval precision reward**. $r_{\text{retrieve}} = \text{Relevance}(d_{\text{retrieved}}, \text{task})$, estimated by a learned relevance model.
- **Memory efficiency reward**. Penalize unnecessary writes: $r_{\text{write}} = -\lambda \cdot \mathbf{1}[\text{write}]$, encouraging selective storage.
- **Consistency reward**. Reward memory states that are internally consistent (no contradictions).

The combined reward for a memory operation m_t at step t :

$$r_t^{\text{mem}} = r_t^{\text{task}} + \alpha \cdot r_t^{\text{retrieve}} + \beta \cdot r_t^{\text{write}} + \gamma \cdot r_t^{\text{consistency}}.$$

17.7.2 Learning What to Remember

The *what-to-remember* problem is a meta-learning challenge: the agent must learn a write policy $\pi_{\text{write}}(e)$ that maximizes future task performance. This is difficult because:

1. The value of a memory is only revealed in the future (delayed reward).
2. The space of possible future queries is unknown at write time.
3. Memories interact: the value of storing e depends on what else is in $\mathcal{M}$.

Approaches:

- **Hindsight relabeling** [8]. After a successful episode, retroactively label the memories that were retrieved as “important” and train the write policy to store similar items.
- **Meta-RL** [79]. Train the write policy across a distribution of tasks; the policy learns to store information that generalizes across tasks.
- **Curiosity-driven storage** [286]. Store observations that are surprising (high prediction error), as these are likely to be informative.

17.7.3 Memory-Augmented Policy Optimization

The idea of jointly optimizing a policy and its memory system dates to differentiable memory networks [119] and was extended to retrieval-augmented LLMs by REALM [128]. The full policy gradient objective for a memory-augmented agent:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right] - \lambda \cdot \mathcal{L}_{\text{mem}}(\phi),$$

where θ are the LLM parameters, ϕ are the memory system parameters (e.g. retrieval model weights), and $\mathcal{L}_{\text{mem}}$ is a regularization term on memory complexity.

Key Insight: Memory as a Learned Inductive Bias

Training memory operations with RL allows the agent to develop task-specific memory strategies. A coding agent learns to store API signatures; a research agent learns to store citation chains; a customer service agent learns to store user complaint patterns. The memory system becomes a *learned inductive bias* tailored to the agent’s domain.

17.8 Comparison of Memory Approaches

Table 17.2: Comparison of agentic memory architectures across key dimensions.

Architecture	Capacity	Retrieval	Update Cost	Trainable	Best For
In-context (working)	$O(L)$ tokens	0 ms	Free	Via fine-tuning	Short tasks, active reasoning
Dense RAG [208]	$O(10^7)$ docs	10–50 ms	$O(1)$ embed	Encoder only	Semantic search, QA
Sparse (BM25) [309]	$O(10^8)$ docs	1–5 ms	$O(d)$ index	No	Keyword search, legal/medical
Hybrid RAG [44]	$O(10^7)$ docs	15–60 ms	$O(1)$ embed	Encoder only	General-purpose retrieval
Summarization	Unlimited	0 ms (in-ctx)	$O(e)$ LLM call	Via fine-tuning	Long conversations, narratives
Knowledge Graph [197]	$O(10^9)$ triples	5–100 ms	$O(1)$ insert	Embedding layer	Structured facts, multi-hop
KV Memory Net [342]	$O(M)$ slots	$O(M)$ attn	Gradient step	Fully	End-to-end differentiable tasks
MemGPT tiered [281]	Unlimited	0–100 ms	Mixed	Via RL	Long-horizon agents, assistants
Graph RAG [82]	$O(10^7)$ nodes	20–200 ms	$O(1)$ insert	Encoder only	Complex reasoning, communities

17.9 Evaluating Memory Systems

Evaluating agentic memory is challenging because the quality of memory operations is only revealed *indirectly*—through downstream task performance over long horizons. A memory system that achieves perfect recall of stored facts can still fail if it retrieves irrelevant context or overwhelms the LLM’s context window.

17.9.1 Evaluation Dimensions

LongMemEval [384] identifies five core capabilities that a long-term memory system must demonstrate:

1. **Information extraction.** Can the system identify and store salient facts from conversational turns? Measured by fact recall: what fraction of ground-truth facts are recoverable from memory?
2. **Multi-session reasoning.** Can the system synthesize information scattered across multiple past sessions? E.g., “Based on our conversations last week and yesterday, what changed in the project scope?”
3. **Temporal reasoning.** Can the system correctly answer time-dependent queries? E.g., “What did I say was my priority *before* the reorg?” requires distinguishing temporal states.
4. **Knowledge updates.** When facts change (user moves cities, preferences shift), does memory reflect the latest state while preserving history?
5. **Abstention.** When the system has no relevant memory, does it correctly say “I don’t know” rather than hallucinate a plausible but fabricated recollection?

17.9.2 Benchmarks

Table 17.3: Benchmarks for evaluating agentic memory systems.

Benchmark	Venue	Scale	Focus
LongMemEval [384]	ICLR 2025	500 questions, scalable histories	Five memory abilities; multi-session chat
LOCOMO [247]	EMNLP 2024	Multi-session dialogues	Single-hop, temporal, multi-hop, open-domain QA over conversations
InfiniteBench [428]	ACL 2024	100K+ token contexts	Long-context recall, not memory-specific but tests limits

17.9.3 Metrics

Memory-Level Metrics.

- **Memory Recall:** $\frac{\# \text{ ground-truth facts retrievable from memory}}{\# \text{ total ground-truth facts}}$. Measures completeness of storage.
- **Memory Precision:** $\frac{\# \text{ relevant items in top-}k \text{ retrieval}}{k}$. Measures noise in retrieval.
- **Latency:** time from query to retrieved context (p50 and p95).
- **Token efficiency:** total tokens injected into context per query. Lower is better—unnecessary context degrades LLM accuracy and increases cost.

Downstream Metrics.

- **Answer accuracy:** correctness of the final response conditioned on memory (EM, F1, or LLM-as-judge).
- **Faithfulness:** does the response accurately reflect what memory contains, without fabrication?
- **Personalization quality:** user satisfaction, measured via preference ratings or A/B tests between memory-augmented and memoryless systems.
- **Contradiction rate:** how often the system produces responses inconsistent with previously stated facts.

Operational Metrics.

- **Write selectivity:** fraction of turns that trigger a memory write. Too high → noise; too low → gaps.
- **Staleness:** how often outdated facts are retrieved despite an update existing.
- **Storage growth rate:** tokens stored per interaction hour. Unbounded growth is unsustainable.

The Evaluation Gap

Most memory papers evaluate on short benchmarks (10–50 sessions). Real production agents run for months with thousands of sessions. Long-horizon evaluation—where memory drift, contradiction accumulation, and storage bloat become dominant failure modes—remains an open challenge. Practitioners should complement benchmark scores with longitudinal monitoring of operational metrics.

17.10 Implementation Patterns

17.10.1 Vector Store Memory with Embeddings

The most common memory pattern stores entries as embedding vectors alongside metadata (timestamps, importance scores, tags). Retrieval combines cosine similarity with temporal decay, so recent and important memories surface first. Duplicate detection and LRU eviction keep the store bounded.

```
import numpy as np
from dataclasses import dataclass, field
from datetime import datetime
from typing import Optional
import json

@dataclass
class MemoryEntry:
    """A single memory entry with metadata."""
    content: str
    embedding: np.ndarray
    timestamp: datetime = field(default_factory=datetime.now)
    importance: float = 0.5
    access_count: int = 0
    last_accessed: Optional[datetime] = None
    tags: list[str] = field(default_factory=list)
    source: str = "agent"

class VectorMemoryStore:
    """
    Hybrid dense+sparse memory store with temporal decay.
    Supports importance-weighted retrieval and LRU eviction.
    """

    def __init__(
        self,
        embed_fn,  # callable: str -> np.ndarray
        max_entries: int = 10_000,
        decay_rate: float = 0.01,  # per hour
        recency_weight: float = 0.3,
    ):
        self.embed_fn = embed_fn
        self.max_entries = max_entries
        self.decay_rate = decay_rate
        self.recency_weight = recency_weight
        self.entries: list[MemoryEntry] = []

# -- Write -----
```

```

def write(
    self,
    content: str,
    importance: float = 0.5,
    tags: list[str] | None = None,
    check_duplicates: bool = True,
) -> MemoryEntry:
    """Commit a new memory, evicting if at capacity."""
    if check_duplicates and self._is_duplicate(content):
        return None # Skip near-duplicate entries

    embedding = self.embed_fn(content)
    entry = MemoryEntry(
        content=content,
        embedding=embedding,
        importance=importance,
        tags=tags or [],
    )

    if len(self.entries) >= self.max_entries:
        self._evict()

    self.entries.append(entry)
    return entry

def _is_duplicate(self, content: str, threshold: float = 0.95) -> bool:
    """Check if a near-duplicate already exists."""
    if not self.entries:
        return False
    emb = self.embed_fn(content)
    sims = self._cosine_similarities(emb)
    return float(np.max(sims)) > threshold

def _evict(self):
    """Remove the least important + least recent entry."""
    now = datetime.now()
    scores = []
    for e in self.entries:
        age_hours = (now - e.timestamp).total_seconds() / 3600
        recency = np.exp(-self.decay_rate * age_hours)
        score = e.importance * (1 - self.recency_weight) \
            + recency * self.recency_weight
        scores.append(score)
    worst_idx = int(np.argmin(scores))
    self.entries.pop(worst_idx)

# -- Retrieve -----

def retrieve(
    self,
    query: str,
    k: int = 5,
    recency_boost: bool = True,
) -> list[MemoryEntry]:
    """
    Hybrid retrieval: dense similarity + temporal recency.
    Returns top-k entries sorted by combined score.
    """
    if not self.entries:
        return []

    q_emb = self.embed_fn(query)
    dense_scores = self._cosine_similarities(q_emb)

    now = datetime.now()
    combined = []

```

```

for i, (entry, d_score) in enumerate(
    zip(self.entries, dense_scores)
):
    if recency_boost:
        age_h = (now - entry.timestamp).total_seconds() / 3600
        recency = np.exp(-self.decay_rate * age_h)
        score = (1 - self.recency_weight) * d_score \
            + self.recency_weight * recency
    else:
        score = d_score
    combined.append((score, i))

combined.sort(reverse=True)
top_k = [self.entries[i] for _, i in combined[:k]]

# Update access metadata
for entry in top_k:
    entry.access_count += 1
    entry.last_accessed = now

return top_k

def _cosine_similarities(self, query_emb: np.ndarray) -> np.ndarray:
    """Vectorized cosine similarity against all stored embeddings."""
    matrix = np.stack([e.embedding for e in self.entries])
    norms = np.linalg.norm(matrix, axis=1, keepdims=True)
    matrix_norm = matrix / (norms + 1e-8)
    q_norm = query_emb / (np.linalg.norm(query_emb) + 1e-8)
    return matrix_norm @ q_norm

# -- Reflect -----

def reflect(self, llm_fn, k: int = 10) -> list[str]:
    """
    Meta-cognitive reflection: retrieve recent memories,
    synthesize higher-level insights, and store them back.
    """
    if len(self.entries) < 3:
        return []

    # Retrieve recent high-importance memories
    recent = sorted(
        self.entries, key=lambda e: e.timestamp, reverse=True
    )[:k]
    context = "\n".join(f"-- {e.content}" for e in recent)

    # Ask LLM to generate insights
    prompt = (
        "Given these recent memories, extract 2-3 high-level "
        "insights or patterns:\n" + context
    )
    raw_insights = llm_fn(prompt)

    # Store each insight as a high-importance memory
    insights = []
    for line in raw_insights.strip().split("\n"):
        line = line.strip().lstrip("-*").strip()
        if len(line) > 20:
            self.write(
                f"[INSIGHT] {line}",
                importance=0.9,
                check_duplicates=True,
            )
            insights.append(line)
    return insights

```

```

def get_stats(self) -> dict:
    """Return memory statistics for monitoring."""
    return {
        "total_entries": len(self.entries),
        "avg_importance": float(
            np.mean([e.importance for e in self.entries])
        ) if self.entries else 0.0,
        "oldest_entry": min(
            (e.timestamp for e in self.entries), default=None
        ),
    }

```

Listing 17.1: Vector store memory with embeddings, importance scoring, and hybrid retrieval.

17.10.2 Hierarchical Memory Manager

Inspired by MemGPT [281], this pattern organises memory into three tiers: *hot* (in-context, immediate access), *warm* (vector store, fast retrieval), and *cold* (archival, unlimited capacity). Entries are automatically promoted or demoted based on access frequency and importance—analogueous to CPU cache hierarchies.

```

from enum import Enum
from collections import OrderedDict

class MemoryTier(Enum):
    HOT = "hot" # In-context: immediate access
    WARM = "warm" # Vector store: fast retrieval
    COLD = "cold" # Archival: slow but unlimited

class HierarchicalMemoryManager:
    """
    Three-tier memory manager inspired by MemGPT.
    Hot tier is an LRU cache; warm is a vector store;
    cold is append-only archival storage.
    """

    def __init__(
        self,
        vector_store: VectorMemoryStore,
        hot_capacity: int = 20, # max entries in hot tier
        warm_capacity: int = 5_000,
        llm_summarize_fn=None, # callable for summarization
    ):
        self.vector_store = vector_store
        self.hot_capacity = hot_capacity
        self.warm_capacity = warm_capacity
        self.summarize = llm_summarize_fn

        # Hot tier: ordered dict for LRU semantics
        self.hot: OrderedDict[str, MemoryEntry] = OrderedDict()
        # Cold tier: append-only list (would be a DB in production)
        self.cold: list[MemoryEntry] = []

        # -- Page-in: promote warm -> hot -----

    def page_in(self, query: str, k: int = 3) -> list[MemoryEntry]:
        """
        Retrieve from warm store and promote to hot tier.
        Evicts least-recently-used hot entries if needed.
        """
        candidates = self.vector_store.retrieve(query, k=k)
        promoted = []
        for entry in candidates:
            key = entry.content[:64] # use prefix as key
            if key not in self.hot:

```



```

        if len(self.hot) >= self.hot_capacity:
            self._evict_hot()
            self.hot[key] = entry
            self.hot.move_to_end(key)
            promoted.append(entry)
        return promoted

def _evict_hot(self):
    """Evict LRU entry from hot tier back to warm."""
    # OrderedDict: first item is LRU
    key, entry = self.hot.popitem(last=False)
    # Re-insert into warm store (already there, just update access)
    # In a real system, we'd update the warm store's metadata

# -- Write with tier assignment -----

def write(
    self,
    content: str,
    importance: float = 0.5,
    tier: MemoryTier = MemoryTier.WARM,
) -> MemoryEntry:
    """Write to the appropriate tier."""
    if tier == MemoryTier.HOT:
        entry = MemoryEntry(
            content=content,
            embedding=self.vector_store.embed_fn(content),
            importance=importance,
        )
        key = content[:64]
        if len(self.hot) >= self.hot_capacity:
            self._evict_hot()
        self.hot[key] = entry
        return entry

    elif tier == MemoryTier.WARM:
        return self.vector_store.write(content, importance=importance)

    else: # COLD
        entry = MemoryEntry(
            content=content,
            embedding=np.array([]), # no embedding for cold
            importance=importance,
        )
        self.cold.append(entry)
        return entry

# -- Summarize and compress -----

def compress_hot_to_warm(self) -> Optional[str]:
    """
    Summarize hot tier contents and write summary to warm.
    Called when hot tier is full and new important content arrives.
    """
    if not self.hot or not self.summarize:
        return None

    hot_contents = "\n".join(
        f"- {e.content}" for e in self.hot.values()
    )
    summary = self.summarize(
        f"Summarize these memory entries concisely:\n{hot_contents}"
    )
    self.vector_store.write(summary, importance=0.7)
    return summary

```

```

# -- Unified retrieval -----

def retrieve(self, query: str, k: int = 5) -> list[MemoryEntry]:
    """
    Retrieve from all tiers, prioritizing hot.
    Returns up to k entries sorted by relevance.
    """
    results = []

    # 1. Check hot tier (exact + semantic)
    q_emb = self.vector_store.embed_fn(query)
    for entry in self.hot.values():
        if entry.embedding.size > 0:
            sim = float(
                np.dot(q_emb, entry.embedding)
                / (np.linalg.norm(q_emb) * np.linalg.norm(entry.embedding) + 1
e-8)
            )
            if sim > 0.7:
                results.append((sim + 1.0, entry)) # +1 hot bonus

    # 2. Retrieve from warm store
    warm_results = self.vector_store.retrieve(query, k=k)
    for entry in warm_results:
        results.append((0.5, entry))

    # 3. Deduplicate and sort
    seen = set()
    final = []
    for score, entry in sorted(results, reverse=True):
        key = entry.content[:64]
        if key not in seen:
            seen.add(key)
            final.append(entry)
        if len(final) >= k:
            break

    return final

def get_hot_context(self) -> str:
    """Return hot tier as a formatted context string."""
    if not self.hot:
        return ""
    lines = ["[Memory Context]"]
    for entry in list(self.hot.values())[-10:]: # last 10
        lines.append(f" - {entry.content}")
    return "\n".join(lines)

```

Listing 17.2: Hierarchical memory manager implementing hot/warm/cold tiers with automatic promotion and demotion.

17.10.3 Memory-Augmented Agent Loop

This pattern, introduced by MemGPT [281] and formalized in the CoALA framework [344], wires the memory system into the agent's reasoning loop via a *read-act-reflect-write* cycle: before responding, the agent retrieves relevant memories; after responding, it decides what to store. Special tokens in the LLM output trigger memory operations, giving the model self-directed control over its own persistence.

```

import re
from typing import Any

class MemoryAugmentedAgent:
    """

```

```
An LLM agent with a full read-act-reflect-write memory cycle.
Implements the MemGPT-style self-directed memory management.
"""
```

```
SYSTEM_PROMPT = """You are a memory-augmented AI assistant.
You have access to persistent memory across conversations.
```

```
At each turn you may issue memory commands:
```

```
[MEMORY_SEARCH: <query>] - retrieve relevant memories
[MEMORY_WRITE: <content>] - store important information
[MEMORY_REFLECT]         - synthesize insights from memory
```

```
Always think step by step. Use memory to avoid repeating mistakes
and to personalize your responses."""
```

```
def __init__(
    self,
    llm_fn,
    memory_manager: HierarchicalMemoryManager,
    importance_threshold: float = 0.6,
    max_memory_tokens: int = 1500,
):
    self.llm = llm_fn
    self.memory = memory_manager
    self.importance_threshold = importance_threshold
    self.max_memory_tokens = max_memory_tokens
    self.conversation_history: list[dict] = []

# -- Main agent step -----
def step(self, user_message: str) -> str:
    """
    Full agent step:
    1. Retrieve relevant memories
    2. Construct augmented prompt
    3. Generate response (possibly with memory commands)
    4. Execute memory commands
    5. Reflect and consolidate
    6. Return response to user
    """

    # Step 1: Retrieve relevant memories
    memories = self.memory.retrieve(user_message, k=5)
    memory_context = self._format_memories(memories)

    # Step 2: Construct augmented prompt
    messages = self._build_messages(user_message, memory_context)

    # Step 3: Generate response
    raw_response = self.llm(messages)

    # Step 4: Execute any memory commands in the response
    clean_response, memory_ops = self._parse_memory_commands(
        raw_response
    )
    self._execute_memory_ops(memory_ops, user_message, clean_response)

    # Step 5: Auto-write important information
    self._auto_write(user_message, clean_response)

    # Step 6: Update conversation history
    self.conversation_history.append(
        {"role": "user", "content": user_message}
    )
    self.conversation_history.append(
        {"role": "assistant", "content": clean_response}
    )
```

```

        return clean_response

# -- Memory retrieval and formatting -----

def _format_memories(self, memories: list[MemoryEntry]) -> str:
    if not memories:
        return ""
    lines = ["Relevant memories:"]
    for i, m in enumerate(memories, 1):
        age = (datetime.now() - m.timestamp).days
        lines.append(
            f"  [{i}] (importance={m.importance:.1f}, "
            f"{age}d ago) {m.content}"
        )
    return "\n".join(lines)

def _build_messages(
    self, user_message: str, memory_context: str
) -> list[dict]:
    system = self.SYSTEM_PROMPT
    if memory_context:
        system += f"\n\n{memory_context}"
    system += f"\n\n{self.memory.get_hot_context()}"

    messages = [{"role": "system", "content": system}]
    # Include recent conversation history (last 6 turns)
    messages.extend(self.conversation_history[-6:])
    messages.append({"role": "user", "content": user_message})
    return messages

# -- Memory command parsing -----

def _parse_memory_commands(
    self, response: str
) -> tuple[str, list[dict]]:
    """Extract and remove memory commands from response."""
    ops = []
    patterns = {
        "search": r"\[MEMORY_SEARCH:\s*(.+?)\]",
        "write": r"\[MEMORY_WRITE:\s*(.+?)\]",
        "reflect": r"\[MEMORY_REFLECT\]",
    }
    clean = response
    for op_type, pattern in patterns.items():
        for match in re.finditer(pattern, response, re.DOTALL):
            content = match.group(1) if op_type != "reflect" else None
            ops.append({"type": op_type, "content": content})
            clean = clean.replace(match.group(0), "").strip()
    return clean, ops

def _execute_memory_ops(
    self,
    ops: list[dict],
    user_msg: str,
    response: str,
):
    """Execute memory commands issued by the LLM."""
    for op in ops:
        if op["type"] == "search":
            results = self.memory.retrieve(op["content"], k=3)
            # Page results into hot tier for immediate use
            self.memory.page_in(op["content"], k=3)

        elif op["type"] == "write":
            self.memory.write(

```

```

        op["content"],
        importance=0.8, # explicitly written = important
        tier=MemoryTier.WARM,
    )

    elif op["type"] == "reflect":
        self._reflect()

# -- Auto-write heuristic -----

def _auto_write(self, user_msg: str, response: str):
    """
    Automatically store important information without explicit command.
    Uses a simple heuristic: write if response contains facts,
    decisions, or user preferences.
    """
    importance_keywords = [
        "remember", "important", "note that", "you prefer",
        "your name is", "decided to", "the answer is",
        "key insight", "learned that",
    ]
    combined = (user_msg + " " + response).lower()
    if any(kw in combined for kw in importance_keywords):
        summary = f"User: {user_msg[:100]} | Agent: {response[:200]}"
        self.memory.write(
            summary,
            importance=self.importance_threshold,
            tier=MemoryTier.WARM,
        )

# -- Reflection -----

def _reflect(self):
    """
    Meta-cognitive reflection: synthesize insights from recent memory.
    Stores high-level insights back into semantic memory.
    """
    recent = self.memory.retrieve("recent important events", k=10)
    if len(recent) < 3:
        return # Not enough to reflect on

    recent_text = "\n".join(f"- {m.content}" for m in recent)
    insight_prompt = [
        {"role": "system", "content": "You extract high-level insights."},
        {"role": "user", "content":
            f"Based on these memories, what are 2-3 key insights?\n"
            f"{recent_text}\nRespond with bullet points only."},
    ]
    insights = self.llm(insight_prompt)
    # Store each insight as a high-importance semantic memory
    for line in insights.split("\n"):
        line = line.strip().lstrip("*-").strip()
        if len(line) > 20:
            self.memory.write(
                f"[INSIGHT] {line}",
                importance=0.9,
                tier=MemoryTier.WARM,
            )

```

Listing 17.3: Complete memory-augmented agent loop with read-act-reflect-write cycle.

The Read-Act-Reflect-Write Cycle

The memory-augmented agent loop implements a four-phase cognitive cycle:

1. **Read:** Before acting, retrieve relevant memories to inform the response.

2. **Act:** Generate a response conditioned on retrieved context.
3. **Reflect:** Periodically synthesize higher-level insights from accumulated memories.
4. **Write:** Selectively commit important new information to persistent storage.

This cycle mirrors the *observe-orient-decide-act* (OODA) loop from military strategy and the *encode-store-retrieve* model from cognitive psychology. The key insight is that memory is not a passive store but an *active participant* in cognition.

17.11 Recent Advances in Agentic Memory

The memory systems described above established the foundational patterns. Several recent works push the boundaries further:

17.11.1 CoALA: Cognitive Architectures for Language Agents

Sumers et al. [344] propose *Cognitive Architectures for Language Agents* (CoALA), a unifying framework that organizes the growing zoo of LLM agents using principles from cognitive science and symbolic AI. CoALA decomposes a language agent into:

- **Modular memory:** working memory (the context window), episodic memory (past experiences), semantic memory (world knowledge), and procedural memory (action schemas)—mirroring our taxonomy in Section 17.2.
- **Structured action space:** internal actions (reasoning, retrieval, memory writes) and external actions (tool use, environment interaction).
- **Decision cycle:** a generalized sense–plan–act loop with explicit retrieval and write steps.

CoALA’s contribution is less a new system than a *design language*: it provides a systematic way to analyze existing agents and identify missing capabilities, making it a useful reference architecture for practitioners.

17.11.2 Mem0: Production-Scale Memory Layer

Mem0 [53] addresses the gap between research memory systems and production deployment. Key ideas:

- **Automatic extraction:** Rather than relying on the LLM to explicitly issue memory-write commands, Mem0 automatically extracts salient facts from conversation turns and consolidates them into a persistent store.
- **Graph-based memory:** Beyond flat vector stores, Mem0 maintains a *relational graph* over extracted entities and facts, enabling multi-hop memory queries (“What did the user say about topic X in the context of project Y?”).
- **Memory compression:** Redundant or superseded facts are automatically merged, keeping the memory store compact and current.

On the LOCOMO benchmark, Mem0 achieves 26% relative improvement over OpenAI’s baseline memory, with 91% lower p95 latency and >90% token cost reduction compared to full-context approaches.

17.11.3 Sleep-Time Compute: Offline Memory Processing

Lin et al. [218] introduce *sleep-time compute*, a paradigm where agents process and consolidate memory *between* user interactions rather than only at query time. The analogy is to biological sleep, during which the brain consolidates memories and pre-computes useful associations.

How it works. During idle periods (“sleep”), the agent:

1. Anticipates likely future queries given the current context.
2. Pre-computes reasoning chains, summaries, and structured representations.
3. Stores these pre-computed artifacts so that test-time inference can retrieve and reuse them.

Results. Sleep-time compute reduces the test-time compute needed to achieve equivalent accuracy by $\sim 5\times$ on reasoning benchmarks. When amortized across multiple related queries about the same context, average cost per query drops by $2.5\times$. The approach is most effective when user queries are *predictable*—i.e., when the context strongly constrains what questions will be asked.

Memory Consolidation as Offline RL

Sleep-time compute can be viewed as *offline policy improvement*: during idle time, the agent improves its memory representations (policy) using the data it has already collected (past interactions), without new environment interactions. This connects to offline RL methods (Chapter 8) where the agent learns from a static dataset of trajectories.

17.11.4 A-MEM: Zettelkasten-Inspired Agentic Memory

A-MEM [400] introduces a memory system that borrows from the *Zettelkasten* method—a note-taking system based on densely interconnected atomic notes—to enable dynamic, self-organizing memory for LLM agents.

Key Design Principles.

- **Structured notes.** Each memory entry is not a raw text chunk but a *note* with multiple structured attributes: a contextual description, keywords, tags, and explicit links to related notes. This metadata enables richer retrieval than embedding similarity alone.
- **Dynamic linking.** When a new memory is added, the system analyzes existing memories to identify semantically meaningful connections and establishes bidirectional links. The result is a *knowledge network* rather than a flat list.
- **Memory evolution.** Critically, adding a new note can *trigger updates* to existing notes—refining their contextual representations and attributes as the agent’s understanding deepens. This makes memory a living structure that improves over time, not a static archive.
- **Agent-driven organization.** Unlike fixed-schema memory systems, A-MEM lets the LLM itself decide how to organize, link, and update memories—making the organizational structure adaptive to the task domain.

Results. Across six foundation models on multi-session reasoning tasks, A-MEM consistently outperforms flat vector stores, summarization-based memory, and graph-database approaches, demonstrating that *how* memories are organized matters as much as *what* is stored.

17.12 Proactive Memory Architectures

A fundamental challenge in long-horizon agentic tasks is **behavioral state decay**: after dozens of tool calls, the executor agent loses track of its original goals, constraints, and accumulated context. Standard memory systems address this reactively—the agent queries memory when it needs information. **Proactive memory architectures** [253] (Meta AI, July 2026, “Remember When It Matters”) take a different approach: a dedicated memory agent *monitors* the executor and injects constraints *before* the executor drifts.

The Proactive Memory Agent Architecture

The system consists of two decoupled agents:

1. **Executor Agent:** Handles task execution—tool calls, reasoning, output generation. Maintains a lean context focused on the immediate subtask.
2. **Memory Agent:** Runs in parallel, monitoring the executor’s action stream. Maintains a compressed representation of:
 - Long-term goals and success criteria
 - Active constraints (safety rules, user preferences, resource limits)
 - Key decisions made earlier in the trajectory

The memory agent **injects reminders** into the executor’s context only when it detects drift—when the executor’s recent actions suggest it has forgotten a constraint or is moving away from the stated goal. This is the key distinction from reactive memory: the memory agent acts *proactively*, before errors compound.

Proactive Memory: Empirical Results

On long-horizon agentic benchmarks (50+ tool calls per task):

- **Dramatically higher success rates** on tasks requiring sustained constraint adherence (e.g., “never modify files in /prod”, “always cite sources”).
- **~40% API cost increase:** The memory agent adds LLM calls for monitoring and injection. This is the primary trade-off.
- **Graceful degradation:** Without the memory agent, executor success rates drop sharply after ~30 tool calls. With it, performance remains stable to 100+ calls.

RL Connection: Memory Agent as State Summarizer

The proactive memory architecture has a natural interpretation in the RL framework:

- The **memory agent** functions as a **state summarizer**—analogous to the attention mechanism in an actor-critic architecture, which selectively weights relevant past states for the value function.
- In a POMDP (Partially Observable MDP), the memory agent maintains the **belief state**: a compressed sufficient statistic of the history relevant to future decisions.
- The injection mechanism is analogous to **privileged information** in asymmetric actor-critic training: the memory agent has access to the full trajectory history, while the executor operates on a compressed context.
- The 40% cost increase reflects the fundamental trade-off between **Markov state representation** (expensive but sufficient) and **truncated context** (cheap but lossy).

This architecture suggests a promising direction for RL training: jointly optimize the executor policy and the memory agent’s injection policy, where the memory agent learns *when* and *what* to inject to maximize task success.

17.13 Summary

Agentic memory systems are a foundational component of capable AI agents, addressing the fundamental limitation of finite context windows. We have surveyed:

- A **four-way taxonomy** (working, episodic, semantic, procedural) that mirrors cognitive science and reflects distinct engineering requirements.
- **Five architectural families**: RAG-based, summarization-based, graph-based, key-value networks, and tiered virtual context (MemGPT).
- **Four core operations**: write (with importance scoring and contradiction detection), read/retrieve (with temporal decay and query expansion), update (with conflict resolution and consolidation), and reflect (meta-cognitive insight generation).
- **Multi-turn and multi-agent extensions**: user modeling, session continuity, shared memory pools, and blackboard architectures.
- **RL training of memory systems**: reward signals for memory operations, learning what to remember, and memory-augmented policy optimization.
- **Proactive memory**: decoupled memory agents that monitor executor behavior and inject reminders before drift compounds—a shift from reactive retrieval to anticipatory intervention.

The field is rapidly evolving. Key open challenges include: (1) *memory grounding*—ensuring retrieved memories are faithfully incorporated rather than ignored or hallucinated over; (2) *scalable consistency*—maintaining coherent shared memory in large multi-agent systems; (3) *privacy-preserving memory*—enabling personalization without compromising user data; and (4) *proactive injection policies*—learning when and what to remind the executor without overwhelming its context. As context windows grow, the boundary between in-context and external memory will shift, but the fundamental need for *selective, structured, retrievable* information storage will remain.